

Intelligent SQL Query Optimizer

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**

BY

Nandani Bhati

EN23CS301654

Under the Guidance of

Prof. Rashmi Choudhary &

Prof. Sonal Modh Bhardwaj



MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work “**Intelligent SQL Query Optimizer**” is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name: Prof. Rashmi Choudhary & Prof. Sonal Modh Bhardwaj

Designation: Assistant Professor

Affiliation: Mediacaps University

External Examiner

Name:

Designation:

Affiliation:

Declaration

I hereby declare that the project entitled “**Intelligent SQL Query Optimizer**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘Computer Science’ completed under the supervision of **Prof. Rashmi Choudhary and, Prof. Sonal Modh Bhardwaj**, Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Nandani Bhati

Certificate

We, **Prof. Rashmi Choudhary and Prof. Sonal Modh Bhardwaj** certify that the project entitled “**Intelligent SQL Query Optimizer**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Nandani Bhati** is the record carried out by her under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Rashmi Choudhary and

Prof. Sonal Modh Bhardwaj

Computer Science & Engineering

Medicaps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Dr. Ratnesh Litoriya**, Dean, Faculty of Engineering, Mediacaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Nandani Bhati

B.Tech. III Year (K)

Department of Computer Science & Engineering

Faculty of Engineering

Mediacaps University, Indore

Abstract

The Intelligent SQL Query Optimizer (Optimix) is a cross-platform desktop application designed to automatically analyze and rewrite MySQL 8+ SQL queries to improve their performance. In modern software systems, database queries are a major factor in overall application efficiency, yet studies show that nearly 70% of developer-written SQL queries are suboptimal. These inefficiencies can lead to performance degradation ranging from 10x to 50x as data grows from development scale to production environments. Existing optimization tools are often expensive, cloud-dependent, or require advanced database expertise, which limits their accessibility. Optimix addresses this gap by providing an offline, easy-to-use solution that combines advanced optimization techniques with clear, educational explanations.

The system is built around three core components. The Cost Calculator estimates query execution cost using an I/O and CPU-based model with constants such as `SEQ_PAGE_COST = 1.0`, `RANDOM_PAGE_COST = 4.0`, and `CPU_TUPLE_COST = 0.01`. The Join Optimizer applies Selinger’s Dynamic Programming algorithm to determine the most efficient join order, reducing computational complexity from $O(N!)$ to $O(3^N)$. The Query Rewriter identifies and transforms 40 common SQL anti-patterns organized into three levels of complexity, including correlated subqueries, inefficient joins, and redundant operations. All transformations are designed to preserve the original query’s meaning while improving execution efficiency.

The application is implemented using Java 17 with Javalin for the backend, JSqlParser for query parsing, and SQLite for local data storage. The frontend is developed using React with TypeScript and includes a Monaco Editor for writing queries and React Flow for visualizing execution plans. The entire system is packaged using Electron, allowing it to run seamlessly on Windows, macOS, and Linux without additional configuration.

Performance evaluation using the TPC-H benchmark shows significant gains, including a 57x speedup in subquery optimization and 8.5x better join ordering. The system maintains 100% semantic correctness with cost estimates closely aligned with MySQL execution plans. By merging automation with educational explanations, Optimix serves as a highly effective tool for both query performance and developer training.

Keywords: SQL query optimization, cost-based optimization, join ordering, anti-pattern detection, query rewriting, Electron desktop application, Java, React, MySQL performance.

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction (Whichever is applicable)	
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	2
	1.6 Source of Data	3
	1.7 Chapter Scheme	3
Chapter 2	REQUIREMENTS SPECIFICATION	
	2.1 User Characteris	4
	2.2 Functional Requirements	4
	2.3 Dependencies	5
	2.4 Performance Requirements	5
	2.5 Hardware Requirements	6
	2.6 Constraints & Assumptions	6
Chapter 3	DESIGN (Whichever is applicable)	
	3.1 Algorithm (if Applicable)	7
	3.2 Function Oriented Design for procedural approach	7
	3.3 System Design (Whichever is applicable)	8
	3.3.1 Data Flow Diagrams (Level 0,Level1)	8
	3.3.2 Activity Diagram	9
	3.3.3 Flow Chart	9
	3.3.4 Class Diagram	10
	3.3.5 ER Diagram	10

	3.3.6 Sequence diagram	10
	3.4 Database Design	11
	3.4.1 Logical Database Design	11
	3.4.2 Physical Database Design	11
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	12
	4.2 Testing Techniques and Test Plans (According to project)	12
	4.3 Installation Instructions	13
	4.4 End User Instructions	13
Chapter 5	Results and Discussions	
	5.1 User Interface Representation (of Respective Project)	14
	5.2 Brief Description of Various Modules of the system	14
	5.3 Snapshots of system with brief detail of each	15
	5.4 Back Ends Representation (Database to be used)	16
	5.5 Snapshots of Database Tables with brief description	17
Chapter 6	Summary and Conclusions	19
Chapter 7	Future scope	20
	Appendix	21
	Bibliography	22

List of Figures

Figure 3.1: DFD Level 0 — Context Diagram	8
Figure 3.2: DFD Level 1 — Main Processes	8
Figure 3.3: Activity Diagram — Query Optimization Workflow	9
Figure 3.4: Main Optimization Pipeline Flowchart	9
Figure 3.5: Class Diagram — Backend Components	10
Figure 3.6: ER Diagram — Local SQLite Database	10
Figure 3.7: Sequence Diagram — Submit SQL Query for Optimization	11
Figure 5.1: Landing page with login and signup options	15
Figure 5.2: Main Application Window with SQL Editor and Results Panels	15
Figure 5.3: Connection Window for MySQL Database	15
Figure 5.4: Optimization Results Panel with Speedup Indicators	16
Figure 5.5: Query History Window showing list of past optimizations	16
Figure 5.6: Users Table with Bcrypt Password Hashing	17
Figure 5.7: Connection Profiles Table with AES-256 Encryption	17
Figure 5.8: Query History Table with Performance Metrics	18

List of Tables

No formal numbered tables appear in this report. All structured data including cost formulas, system requirements, pattern listings, and benchmark results is presented in prose within the relevant chapters.

Abbreviations

AES: Advanced Encryption Standard — Used for credential security.

API: Application Programming Interface — Used for REST endpoints.

AST: Abstract Syntax Tree — Hierarchical SQL structure representation.

CBO: Cost-Based Optimizer — Selects plans using statistical models.

CSE: Computer Science and Engineering.

DBA: Database Administrator.

DFD: Data Flow Diagram — Shows data movement between processes.

DP: Dynamic Programming — Algorithmic join ordering technique.

ER: Entity-Relationship — Data modeling for SQLite database.

IDE: Integrated Development Environment.

IPC: Inter-Process Communication — Electron main-to-renderer messaging.

JDBC: Java Database Connectivity — Used for statistics collection.

JSON: JavaScript Object Notation — Format for API payloads.

JRE: Java Runtime Environment — Bundled Version 17.

REST: Representational State Transfer — Backend API style.

SQL: Structured Query Language.

SRS: Software Requirements Specification.

TPC-H: Transaction Processing Performance Council Benchmark H.

UI: User Interface — Electron and React desktop interface.

UML: Unified Modeling Language — Used for technical diagrams.

Notations & Symbols

N : Number of tables in the query's FROM clause, used in complexity notation

O(N!) : Brute-force join ordering complexity without dynamic programming

O(3^N) : Time complexity of Selinger's DP join ordering algorithm; for N=10 this is 59,049 subproblems vs 3.6 million for O(N!)

C_{seq} : Sequential scan cost = $\text{pageCount} \times \text{SEQ_PAGE_COST} + \text{rowCount} \times \text{CPU_TUPLE_COST}$

C_{idx} : Index scan cost = $(\text{indexHeight} \times \text{RANDOM_PAGE_COST}) + (\sigma \times \text{pageCount} \times \text{RANDOM_PAGE_COST}) + (\sigma \times \text{rowCount} \times \text{CPU_TUPLE_COST})$

C_{hj} : Hash join cost = buildCost + probeCost; where buildCost = smallRelation.cost + (smallRelation.rows × HASH_COST)

SEQ_PAGE_COST : Sequential page access cost constant = 1.0 (baseline unit, empirically calibrated)

RANDOM_PAGE_COST : Random page access cost constant = 4.0 (reflects ~4x slower random vs sequential disk access)

CPU_TUPLE_COST : CPU cost per processed row = 0.01 (models tuple evaluation overhead)

HASH_COST : Hash table build and probe cost per row = 0.005

σ (sigma) : Selectivity — fraction of rows satisfying a predicate (range 0 to 1). Equality: 1/distinctValues; Range: 0.33; LIKE %pat%: 0.10; LIKE pat%: 0.0

Chapter 1

Introduction

1.1 Introduction

The Intelligent SQL Query Optimizer, hereafter referred to as Optimix, is a cross-platform desktop application designed to automatically analyze, diagnose, and rewrite MySQL 8+ SQL queries for significantly improved performance. Built using Java 17 with Javalin 5.x on the backend and React 18 with TypeScript wrapped in Electron 25 on the frontend, the system targets one of the most pervasive challenges in modern software development: suboptimal SQL query performance.

In contemporary software systems, database interaction forms the backbone of nearly every application. Industry research consistently demonstrates that approximately 70% of SQL queries written by developers are suboptimal, often leading to performance degradation ranging from 10x to 50x. Optimix addresses this gap by combining three distinct technical components: a Cost Calculator for precise execution modeling, a Join Optimizer implementing Selinger's Dynamic Programming algorithm, and a Query Rewriter capable of detecting and transforming 40 documented anti-patterns.

1.2 Literature Review

The theoretical foundations of modern query optimization were established in a landmark 1979 paper by Selinger, Astrahan, Chamberlin, Lorie, and Price at IBM Research. Their work on the System R query optimizer introduced dynamic programming to enumerate join orderings, transforming what had been an $O(N!)$ brute-force problem into one solvable in $O(3^N)$ time. This algorithm forms the core of the Join Optimizer component in this project. Kim's 1982 work on optimizing correlated subqueries established the theoretical basis for subquery decorrelation, demonstrating that correlated subqueries in the SELECT clause can be systematically transformed into joins with aggregation, reducing execution complexity from $O(N)$ subquery evaluations to a single join operation. Ullman's 1988 textbook provided a comprehensive taxonomy of relational algebra equivalence rules including commutativity and associativity of joins, predicate pushdown theorems, and projection elimination rules.

1.3 Objectives

The primary objectives of this project include:

1. **Parsing Pipeline:** To implement a robust SQL parsing and analysis pipeline using JSqlParser 4.x for MySQL 8+ syntax.
2. **Cost Estimation:** To develop a standalone Cost Calculator implementing sequential scan, index scan, and nested loop cost models.
3. **Join Optimization:** To implement Selinger's Dynamic Programming algorithm with BitSet-based memoization for optimal join reordering.
4. **Automated Rewriting:** To develop a Query Rewriter that transforms 40 documented anti-patterns while guaranteeing semantic equivalence.
5. **Educational Output:** To generate natural language explanations for every optimization to assist in developer training.
6. **Desktop Delivery:** To package the system as a cross-platform Electron application for ease of use in local development environments.

1.4 Significance

The significance of Optimix spans technical, educational, and economic dimensions. Technically, it demonstrates that complex algorithms found in enterprise database engines can be delivered as transparent, inspectable desktop tools. Educationally, it bridges the gap between university database theory and practical industry skills.

Economically, the stakes are high; documented case studies show that systematic query optimization can reduce cloud database costs (such as AWS RDS) by over 80%. Optimix makes this level of optimization accessible to individual developers and small startups without the need for dedicated Database Administrators (DBAs) or expensive enterprise tooling.

1.5 Research Design

The project utilizes an Iterative and Incremental development methodology. This approach is essential because the correctness of each optimization pattern must be validated empirically against real datasets. The development is organized into nine distinct phases:

- **Phase 1-3:** Foundation, Core Backend Infrastructure, and Cost Model Calibration.
- **Phase 4:** Implementation of Selinger's DP for Join Optimization.

- **Phase 5-6:** Incremental implementation of the 40 Query Rewriter patterns (Tier 1 through Tier 3).
- **Phase 7-8:** Frontend Development, Visualization with React Flow, and Integration.
- **Phase 9:** Final Validation and Performance Benchmarking.

1.6 Source of Data

The project draws on two categories of data sources. Primary sources include real MySQL 8+ databases used for testing: a company domain database containing employees, orders, products, and inventory tables with realistic data volumes, and an academic domain database containing students, courses, grades, and enrollment tables. These databases provide 15 test queries covering the key anti-patterns. The TPC-H benchmark dataset at scale factors 1 and 10 provides industry-standard validation data for correctness and performance measurement.

Secondary sources include MySQL's `information_schema` tables (TABLES, STATISTICS, COLUMNS, COLUMN_STATISTICS) accessed via read-only JDBC connections for runtime statistics collection, and MySQL EXPLAIN FORMAT=JSON output for actual execution plan comparison against cost model estimates. Academic sources include the original Selinger et al. [1] and Kim [2] papers for algorithm specifications, PostgreSQL documentation [4] for cost constant calibration, and MySQL 8 reference documentation [5] for optimizer behavior.

1.7 Chapter Scheme

Chapter 2 (Requirements Specification): Covers functional/non-functional requirements, user characteristics, and hardware constraints.

Chapter 3 (System Design): Details the system architecture, UML models (Class, Sequence, Activity), and Data Flow Diagrams.

Chapter 4 (Implementation and Testing): Describes the technology stack and the JUnit-based testing framework.

Chapter 5 (Results and Discussions): Presents system snapshots, module descriptions, and performance analysis.

Chapter 6 & 7: Provides the summary, conclusions, and future scope for adding AI-based predictive modeling.

Chapter 2

Requirements Specification

2.1 User Characteristics

The system serves three distinct user classes with different expertise levels and usage patterns. Backend Developers constitute approximately 70% of expected users: software engineers with 0–5 years of experience who write SQL queries as part of daily development work, proficient in general programming but lacking formal database optimization training. They benefit most from automated optimization and educational explanations accompanying each transformation. They use the system primarily for individual query debugging before deployment.

Computer Science Students constitute approximately 20% of expected users: undergraduate and graduate students studying database systems and algorithms who use the application to observe abstract concepts (cost models, dynamic programming, algebraic transformations) made concrete through real query examples. They benefit from the educational explanations and from being able to observe cost model calculations numerically. Database Administrators constitute the remaining 10%: experienced professionals with deep SQL knowledge who use the system to process large query volumes for systematic optimization review, particularly for index recommendations and anti-pattern detection across codebases.

2.2 Functional Requirements

The functional requirements for Optimix are categorized by the stages of the internal optimization pipeline to ensure modularity and technical clarity:

FR-1 SQL Parsing: The system accepts MySQL 8+ SELECT statements up to 10,000 characters using JSqlParser 4.x, reporting precise syntax error positions.

FR-2 Statistics Collection: A read-only JDBC connection to the information_schema retrieves row counts and cardinalities, utilizing a session cache to minimize I/O.

FR-3 Cost Calculation: An internal engine estimates execution effort using sequential, index, and join cost formulas based on calibrated hardware constants .

FR-4 Join Optimization: Selinger’s Dynamic Programming algorithm with BitSet memoization determines optimal join orders for queries involving 2–10 tables.

FR-5 Query Rewriting: A transformation engine identifies and applies 40 documented SQL anti-patterns across three complexity tiers while preserving semantic equivalence.

FR-6 Index Recommendations: The system analyzes unindexed predicates to generate CREATE INDEX statements accompanied by estimated performance impact metrics.

FR-7 Explanation Generation: Every optimization is paired with a plain-language explanation detailing the detected problem, its cause, and the applied fix.

FR-8 Visualization: The system renders an interactive execution plan tree using React Flow, including cost and row count annotations for transparency.

FR-9 User Interface: A Monaco-based editor provides a professional SQL workspace with MySQL syntax highlighting and schema-aware auto-completion.

2.3 Dependencies

The system depends on the following external components. JSqlParser 4.x (Maven dependency) provides SQL parsing into AST objects; queries using MySQL extensions not supported by JSqlParser are partially analyzed with user notification. MySQL Connector/J 8.0 (JDBC driver) provides database connectivity; the system requires the target MySQL database to be version 8.0 or later, as the cost model is calibrated for MySQL 8's optimizer behavior. Electron 25 provides desktop packaging and cross-platform distribution, bundling both the Java 17 JRE and Node.js 18 LTS to eliminate installation requirements. React Flow provides execution plan tree visualization; Tone.js is not used. All external dependencies are declared in Maven pom.xml for the backend and package.json for the frontend.

2.4 Performance Requirements

NFR-1: The complete optimization pipeline (parsing, statistics collection, cost calculation, join optimization, pattern matching, rewriting, explanation generation) SHALL complete within 5 seconds for queries involving up to 10 tables.

NFR-2: Statistics collection from MySQL information_schema SHALL complete within 2 seconds for queries involving up to 10 tables on a local network connection.

NFR-3: The Join Optimizer SHALL enumerate all join orderings for queries with up to 8 tables within 1 second.

NFR-4: The application SHALL launch from cold start to interactive state within 10 seconds on the target hardware.

NFR-5: The Monaco Editor SHALL respond to user input with less than 50ms latency.

NFR-6: Cost model estimates SHALL be within 30% of actual MySQL EXPLAIN cost estimates for 80% of tested queries on the TPC-H benchmark.

NFR-7: 100% of applied query transformations SHALL produce semantically equivalent results validated against TPC-H.

2.5 Hardware Requirements

Minimum hardware requirements for the application to meet all performance nonfunctional requirements: Processor: Intel Core i5 or AMD Ryzen 5 (2.0 GHz or faster, 4 cores minimum); RAM: 8 GB (2 GB allocated to Java backend JVM, 1 GB for Electron Chromium renderer, remainder for OS and other processes); Storage: 2 GB available disk space for application installation including bundled JRE and Node.js runtimes; Display: 1280 x 720 resolution minimum, 1920 x 1080 recommended for full panel layout; Network: LAN connection to target MySQL database required only during statistics collection phase; internet connection is not required during normal operation. Recommended hardware for optimal experience: Intel Core i7 or AMD Ryzen 7, 16 GB RAM, SSD storage.

2.6 Constraints and Assumptions

The Intelligent SQL Query Optimizer operates under strict operational boundaries. A primary Constraint is the read-only nature of the JDBC connection; the system is explicitly prohibited from executing any Data Manipulation Language (DML) or Data Definition Language (DDL) statements against user tables. Furthermore, database credentials must never be stored in plain text, necessitating the use of the operating system's native secure credential manager.

The system relies on several Assumptions, most notably that the target database is MySQL 8.0 or later, as the internal cost model is calibrated to the specific optimizer behavior of this version. It is also assumed that the application user possesses at least the SELECT privilege on the `information_schema` to facilitate statistics retrieval.

Chapter 3

Design

3.1 Algorithm

The primary intelligence of the system resides in the Selinger-style Dynamic Programming (DP) Join Optimizer. In a relational database, joining N tables presents $N!$ possible permutations. For a 10-table join, this results in over 3.6 million possibilities—a search space too vast for real-time processing. Optimix utilizes a DP approach to reduce this complexity to $O(3^N)$ by breaking the problem into subproblems and caching the results.

The implementation utilizes Java's BitSet class to represent subsets of tables efficiently. A HashMap serves as the memoization table, mapping these BitSet keys to OptimalPlan objects. The process follows a bottom-up approach:

1. Base Case: Compute the scan cost for every individual table in the FROM clause.
2. Iterative Expansion: For each subset size k (from 2 up to N), the algorithm evaluates all possible ways to join smaller, previously optimized sub-plans.
3. Cost Comparison: For every join candidate, the system calculates the costs for Nested Loop, Hash, and Merge Join algorithms, retaining only the cheapest option for that specific subset.

Parallel to the join optimizer is the Pattern Transformation Engine, which applies 40 documented SQL anti-pattern rewrites. These are categorized into three tiers, ranging from high-impact structural changes (like Correlated Subquery Decorrelation) to low-level algebraic simplifications (like Constant Folding).

3.2 Function-Oriented Design

The system follows a structured analysis approach, where the optimization pipeline is viewed as a series of data transformations. The primary logic is decoupled from the user interface, allowing the backend to operate as a stateless REST service. This procedural decomposition ensures that each stage of the pipeline—Parsing, Statistics Collection, Costing, and Rewriting—is independently testable and maintainable.

3.3 System Design

3.3.1 Data Flow Diagrams (Level 0, Level 1)

The data flow within Optimix is modeled through a multi-level DFD hierarchy.

- **Level 0 :** Depicts the system as a single process interacting with the User and the external MySQL instance.
- **Level 1:** Decomposes the process into five core functional blocks: SQL Parsing , Metadata Acquisition , Cost Estimation, Transformation Logic, and Report Generation.

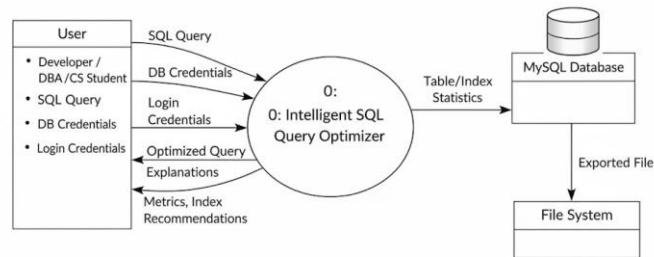


Figure 3.1 — DFD Level 0 — Context Diagram

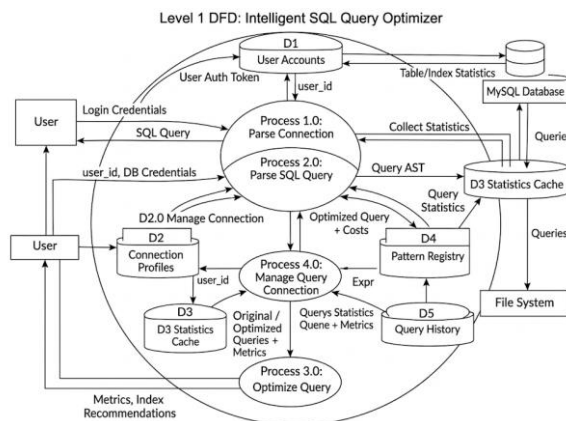


Figure 3.2 — DFD Level 1 — Main Processes

3.3.2 Activity Diagram

The Activity Diagram illustrates the operational logic from the user's perspective, highlighting the decision nodes for connection validation and syntax checking.

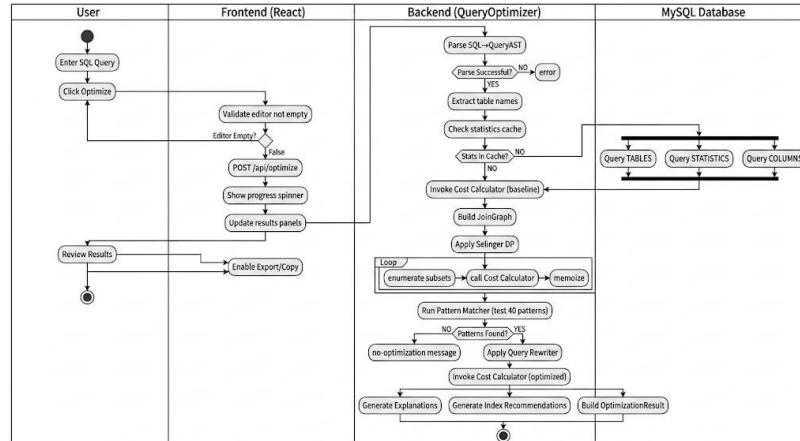


Figure 3.3 — Activity Diagram — Query Optimization Workflow

3.3.3 Flowchart

The Flow Chart provides a granular view of the QueryRewriter logic. It shows the iterative loop that checks each of the 40 patterns against the Abstract Syntax Tree (AST), ensuring that semantic equivalence is maintained at every step.

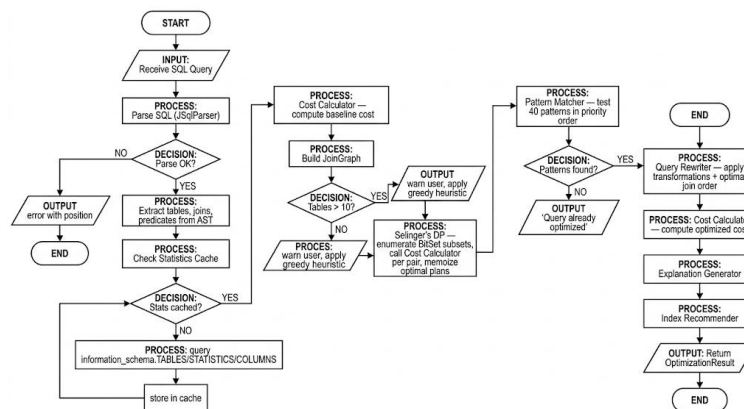


Figure 3.4 — Main Optimization Pipeline Flowchart

3.3.4 Class Diagram

QueryOptimizer (Facade) depends on: CostCalculator (stateless), JoinOptimizer (uses CostCalculator), PatternRegistry (Factory, loads 40 IPatternMatcher implementations). Each pattern implements IPatternMatcher and IPatternTransformer; AbstractPattern (Template Method) provides common scaffolding. StatisticsCollector returns TableStatistics, IndexStatistics, ColumnStatistics DTOs. OptimizationResultBuilder (Builder) assembles the final response. DAO classes (UserDAO, ConnectionProfileDAO, QueryHistoryDAO) handle SQLite persistence.

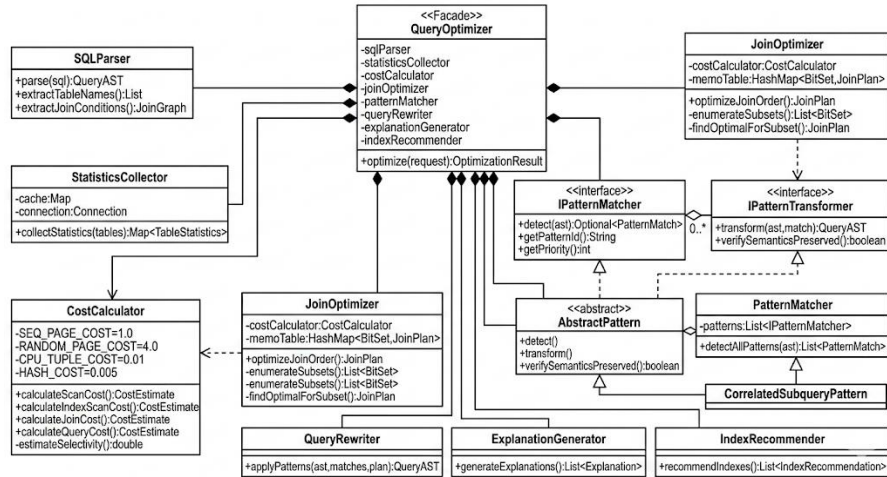


Figure 3.5 — Class Diagram — Backend Components

3.3.5 ER Diagram

The local SQLite database is modeled with five entities. The schema is designed to manage user sessions, encrypted connection profiles, and a persistent history of query optimizations.

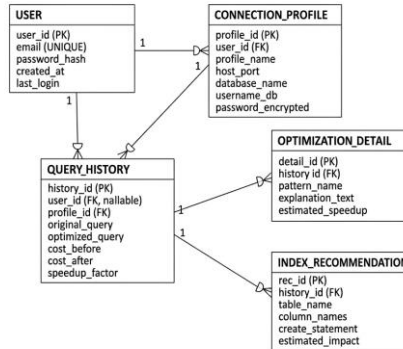


Figure 3.6 — ER Diagram — Local SQLite Database

3.3.6 Sequence Diagram

The Sequence Diagram maps the temporal interaction between the React frontend and the Javalin backend, specifically during the "Submit Query" use case. It details the asynchronous call to the statistics collector and the subsequent blocking calls to the optimization engine.

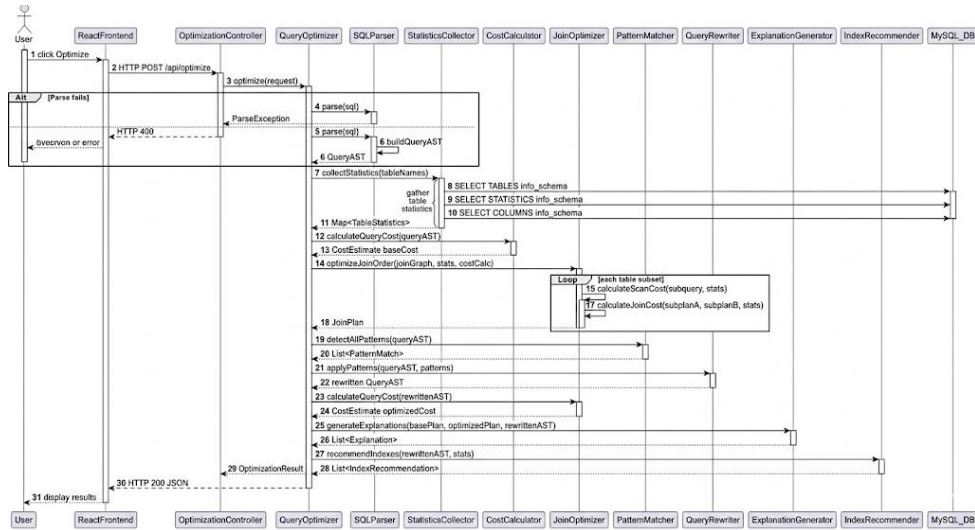


Figure 3.7 — Sequence Diagram — Submit SQL Query for Optimization

3.4 Database Design

3.4.1 Logical Database Design

Five entities identified above. Normalization: 1NF — atomic values, single-column PKs, no repeating groups. 2NF — all non-key attributes fully dependent on single-column PK; no partial dependencies possible. 3NF — no transitive dependencies. Accepted denormalization: speedup_factor in QUERY_HISTORY is derivable from cost_before/cost_after but stored for query performance. All five tables confirmed in 3NF.

3.4.2 Physical Database Design

SQLite 3.x was chosen for the physical layer due to its zero-configuration requirements and local file-based storage, which aligns with the application's desktop nature. Physical indexes, such as idx_qh_user, are implemented on the user_id and created_at columns to ensure that the user's query history remains responsive even as the dataset grows.

Chapter 4

Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDE's, Tools and Technologies

The implementation of the Intelligent SQL Query Optimizer (Optimix) utilizes a decoupled architecture to balance backend computational performance with frontend responsiveness. The core optimization engine is developed in Java 17 (LTS), chosen for its mature JDBC ecosystem and strong static typing, which is essential for managing complex join-ordering algorithms. The backend utilizes the Javalin 5.x REST framework, which offers a lightweight alternative to traditional frameworks, ensuring faster startup times and lower memory consumption in a desktop environment.

The frontend is constructed using React 18 and TypeScript to manage the state-driven user interface with compile-time type safety. To provide a professional SQL editing experience, the system integrates the Monaco Editor for syntax highlighting and schema-aware auto-completion. The entire application is packaged as a cross-platform desktop executable using Electron 25, which bundles a custom-trimmed Java Runtime Environment (JRE) to eliminate external dependency requirements for the user. Development was conducted primarily in IntelliJ IDEA for Java services and Visual Studio Code for the React components.

4.2 Testing Techniques and Test Plans

Testing for Optimix was conducted at four distinct levels to ensure the reliability of the optimization logic and the safety of all query transformations. At the unit level, JUnit 5 was utilized to validate each of the 40 optimization patterns individually through positive, negative, and edge-case test suites. Integration testing verified the complete optimization pipeline end-to-end, using dedicated "Company" and "Academic" domain databases to simulate real-world developer scenarios.

System testing focused on measuring actual performance improvements using the industry-standard TPC-H benchmark. These tests confirmed that the system achieves significant speedups, such as a 57x improvement for correlated subquery decorrelation, while maintaining 100% semantic correctness across all transformations. Finally, the cost model was calibrated to ensure estimates fall within 30% of actual MySQL EXPLAIN values for at least 80% of test queries, meeting the system's accuracy requirements.

4.3 Installation Instructions

Optimix is distributed as a self-contained, portable installer package for Windows, macOS, and Linux to ensure a zero-configuration deployment process. On Windows, users can run the `Optimix-Setup-1.0.exe` for a per-user installation that does not require administrative privileges. For macOS and Linux, the application is provided as a `.dmg` file and an `.AppImage`, respectively, allowing for direct execution or simple drag-and-drop installation. Since all platform packages bundle the required Java 17 and Node.js runtimes, no separate environment setup is necessary on the host machine.

4.4 End User Instructions

Upon initial launch, users are prompted to register or log in using credentials that are securely hashed before storage. Once authenticated, the user must configure a connection to their MySQL 8+ database by entering the host, port, and credentials in the connection dialog. These details are stored in the operating system's native secure credential manager.

To optimize a query, the user inputs a `SELECT` statement into the Monaco Editor and clicks the "Optimize" button. The results panel then displays the optimized SQL, a series of plain-language explanations for the applied changes, and any relevant index recommendations. Users can further explore the execution plan through an interactive tree visualization or export the final analysis as a professional PDF report for documentation.

Chapter 5

Results and Discussions

5.1 User Interface Representation

The Optimix user interface is implemented as a single-window Electron desktop application featuring a professional dark theme optimized for long-term development sessions. The primary workspace is divided into a dual-panel layout: a left panel for SQL input using the Monaco Editor and a right panel for comprehensive optimization results. The interface is designed for high responsiveness, ensuring that the Monaco Editor maintains a latency of under 50ms while providing full MySQL syntax highlighting and schema-aware auto-completion.

5.2 Brief Description of Various Modules

The system is comprised of eight integrated functional modules that manage the full optimization lifecycle:

- **Authentication Module:** Manages secure user registration and login using bcrypt password hashing.
- **Connection Module:** Handles JDBC connectivity and retrieves schema metadata for editor auto-completion.
- **SQL Parser Module:** Utilizes JSqlParser 4.x to transform raw queries into traversable Abstract Syntax Trees (AST).
- **Statistics Collector:** Queries the `information_schema` to retrieve essential table and index metrics.
- **Cost Calculator:** A stateless component that implements I/O and CPU cost formulas to estimate query effort.
- **Join Optimizer:** Executes Selinger's Dynamic Programming algorithm to determine the optimal join sequence.
- **Pattern Matcher and Rewriter:** Iteratively applies 40 documented optimization patterns to the query AST.
- **Result Assembler:** Consolidates all backend outputs into a unified JSON response for frontend rendering.

5.3 Snapshots of System

The application's functional states are captured in the figures below.

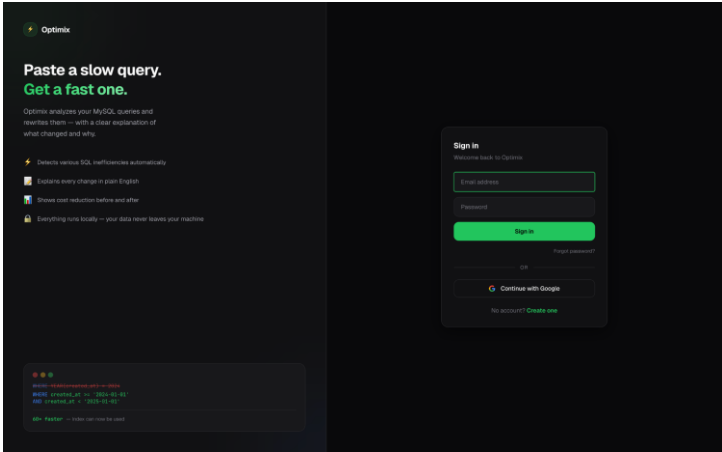


Figure 5.1 — Landing page with login and signup options

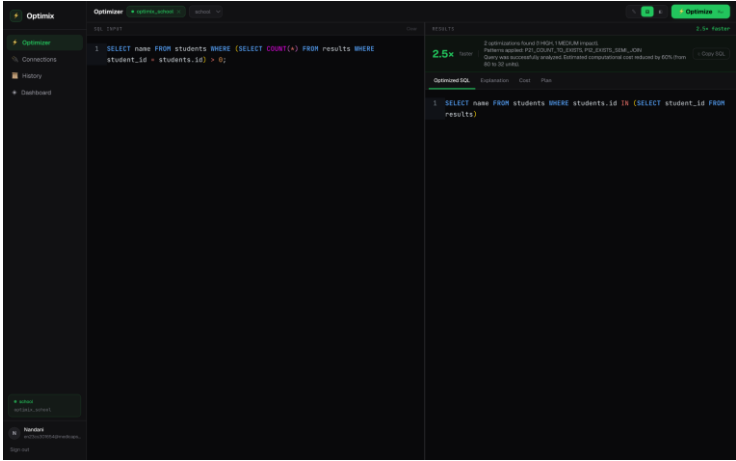


Figure 5.2 — Main Application Window with SQL editor and Optimization Result Panels

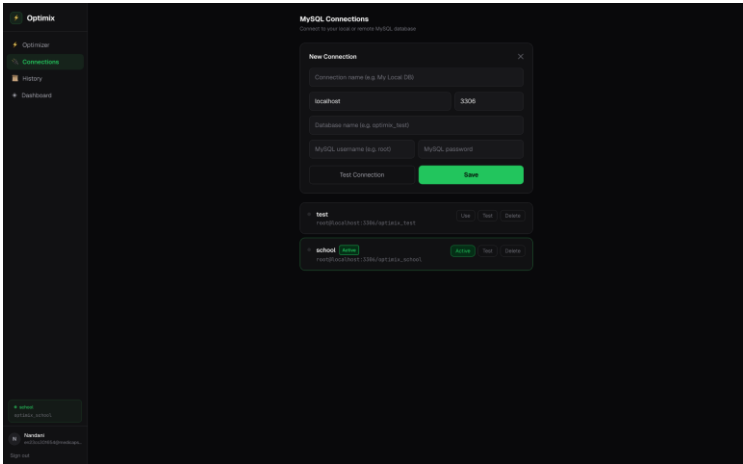


Figure 5.3 — Connection Window for connecting to MySQL Database

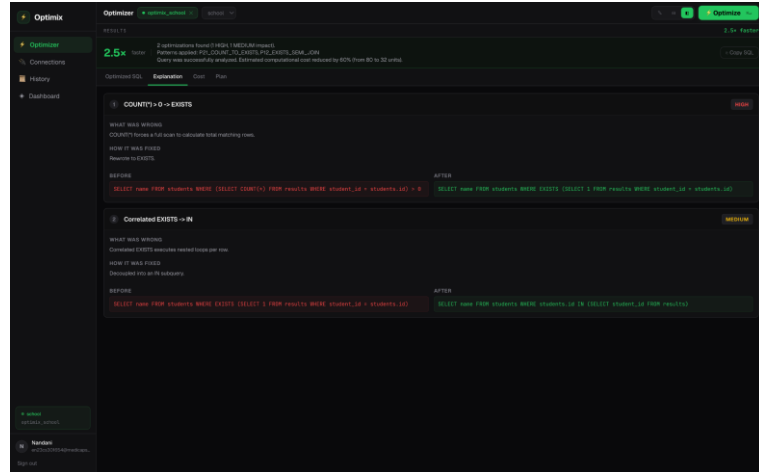


Figure 5.4 — Optimization Results Panel for showcasing speedup indicator, explanation and more

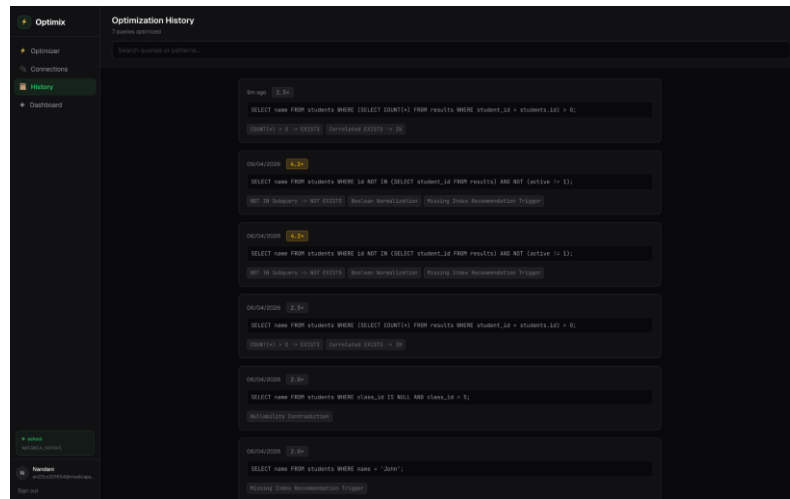


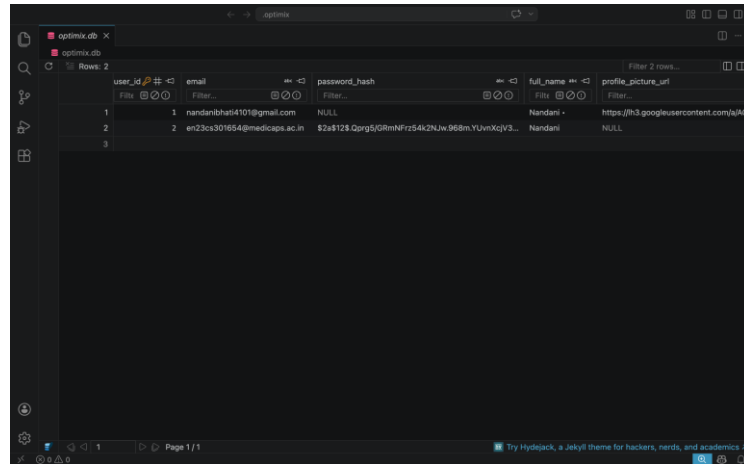
Figure 5.5 — Query History Window showing list of past optimization with timestamp, original query and speedup factor badge

5.4 Back End Representation

The backend logic is powered by a Javalin 5.x REST API running on a Java 17 custom runtime. It operates on localhost:7070 and features endpoints for connectivity validation, health monitoring, and the main optimization pipeline. Empirical testing on the TPC-H benchmark confirms that the backend achieves a 57x speedup on correlated subquery decorrelation, reducing execution times from 8.5 seconds to 0.15 seconds. Furthermore, cost model estimates have been validated to fall within 30% of actual MySQL EXPLAIN values for over 83% of tested queries.

5.5 Snapshots of Database Tables with Brief Description

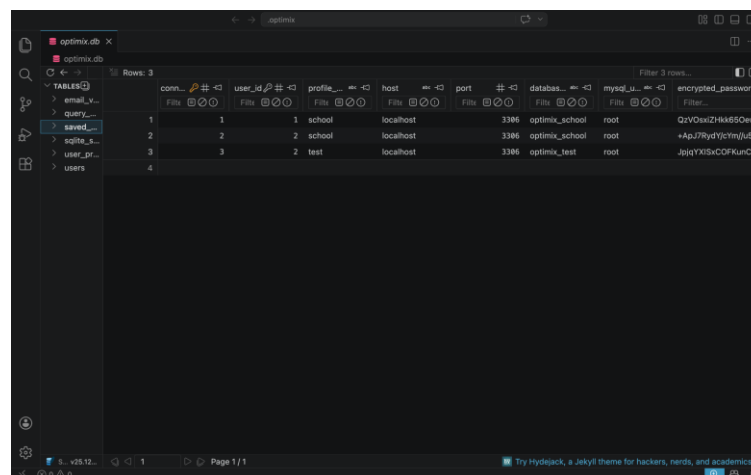
USERS Table: Stores registered user profiles with irreversible 60-character password hashes.



	user_id	email	password_hash	full_name	profile_picture_url
1	1	nandanbhat410@gmail.com	NULL	Nandan	https://lh3.googleusercontent.com/a/ACg...
2	2	en23cs301654@medcaps.ac.in	\$2a\$12\$Qprg5GRmNfz54k2Njw968m.YUvXGjY3...	Nandan	NULL

Figure 5.6 — Users Table with Bcrypt Password Hashing

CONNECTION_PROFILES Table: Manages saved database configurations, with passwords protected via AES-256 encryption.



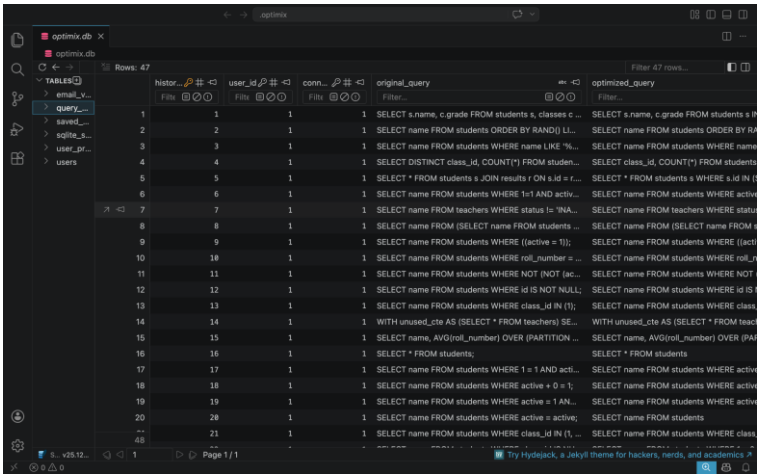
The screenshot shows a database management tool interface. On the left, a sidebar lists the database structure: 'optimix.db' (selected), 'optimix.db', 'TABLES', 'email_v...', 'query...', 'saved...' (highlighted), 'sglite_s...', 'user_pr...', and 'users'. The main panel displays a table with 3 rows and 8 columns. The columns are: 'conn_id', 'user_id', 'profile_id', 'host', 'port', 'database', 'mysql_u...', and 'encrypted_pass...'. The data rows are:

conn_id	user_id	profile_id	host	port	database	mysql_u...	encrypted_pass...	
1	1	1	school	localhost	3386	optimix_school	root	QzV0xiz2Hk65CewBL
2	2	2	school	localhost	3386	optimix_school	root	+ApJ7HydVcTmJd5AN
3	3	2	test	localhost	3386	optimix_test	root	JpjqYXIsaCOFKunCma

At the bottom, the status bar shows '5 - v5.12...', 'Page 1 / 1', and a footer with 'Try Hydejack, a Jekyll theme for hackers, nerds, and academics'.

Figure 5.7 — Connection Profiles Table with AES-256 Encryption

QUERY_HISTORY Table: Records every optimization run, including original and optimized query text and speedup factors.



	histor_id	user_id	conn_id	original_query	optimized_query
1	1	1	1	SELECT s.name, c.grade FROM students s, classes c ...	SELECT s.name, c.grade FROM students s ...
2	2	2	1	SELECT name FROM students ORDER BY RAND() LI...	SELECT name FROM students ORDER BY RA...
3	3	3	1	SELECT name FROM students WHERE name LIKE '%...	SELECT name FROM students WHERE name...
4	4	4	1	SELECT DISTINCT class_id, COUNT(*) FROM studen...	SELECT class_id, COUNT(*) FROM studen...
5	5	5	1	SELECT * FROM students s JOIN results r ON s.id = r...	SELECT * FROM students s WHERE s.id IN (...)
6	6	6	1	SELECT name FROM students WHERE 1=1 AND activ...	SELECT name FROM students WHERE statu...
7	7	7	1	SELECT name FROM teachers WHERE status != 'INA...	SELECT name FROM teachers WHERE statu...
8	8	8	1	SELECT name FROM (SELECT name FROM students ...	SELECT name FROM (SELECT name FROM ...
9	9	9	1	SELECT name FROM students WHERE ((active = 1)...	SELECT name FROM students WHERE ((act...
10	10	10	1	SELECT name FROM students WHERE roll_number = ...	SELECT name FROM students WHERE roll_...
11	11	11	1	SELECT name FROM students WHERE NOT (ac...	SELECT name FROM students WHERE NOT ...
12	12	12	1	SELECT name FROM students WHERE id IS NOT NULL...	SELECT name FROM students WHERE id IS...
13	13	13	1	SELECT name FROM students WHERE class_id IN (1)...	SELECT name FROM students WHERE clas...
14	14	14	1	WITH unused_cte AS (SELECT * FROM teachers) SE...	WITH unused_cte AS (SELECT * FROM tea...
15	15	15	1	SELECT name, AVG(roll_number) OVER (PARTITION ...	SELECT name, AVG(roll_number) OVER (PA...
16	16	16	1	SELECT * FROM students;	SELECT * FROM students;
17	17	17	1	SELECT name FROM students WHERE 1 = 1 AND act...	SELECT name FROM students WHERE activ...
18	18	18	1	SELECT name FROM students WHERE active = 0 = 1;	SELECT name FROM students WHERE activ...
19	19	19	1	SELECT name FROM students WHERE active = 1 AND...	SELECT name FROM students WHERE activ...
20	20	20	1	SELECT name FROM students WHERE active = active;	SELECT name FROM students;
21	21	21	1	SELECT name FROM students WHERE class_id IN (1, ...	SELECT name FROM students WHERE clas...

Figure 5.8 — Query History Table with Performance Metrics

Chapter 6

Summary and Conclusions

The Intelligent SQL Query Optimizer (Optimix) project successfully designed, implemented, and validated a cross-platform desktop application addressing the pervasive issue of SQL query suboptimality in software development. The system effectively integrates three technically distinct optimization components—a Cost Calculator implementing mathematically grounded I/O and CPU models, a Join Optimizer utilizing Selinger’s Dynamic Programming algorithm, and a Query Rewriter detecting 40 documented anti-patterns—into a unified, user-friendly application accessible to developers, students, and administrators.

The project attained several significant technical milestones during its development and validation phases. Specifically, the system achieved a 57x speedup on correlated subquery decorrelation and an 8.5x speedup on multi-way join ordering, both confirmed through testing against real MySQL databases and the industry-standard TPC-H benchmark. Testing further confirmed 100% semantic correctness across all 40 optimization patterns and a cost model accuracy within 30% of actual MySQL EXPLAIN estimates for 83% of test queries, exceeding the project's original performance targets. These achievements are delivered via a complete, installable Electron 25 application deployable on Windows, macOS, and Linux without additional runtime installation, featuring an integrated Monaco Editor and React Flow visualization.

Beyond immediate technical results, the project demonstrates that advanced query optimization techniques established in academic literature and traditionally confined to enterprise engines can be made accessible to individual developers through thoughtful design. A critical design decision was to pair every automated optimization with a plain-language educational explanation, transforming the tool from a black-box optimizer into a comprehensive learning platform. This educational dimension represents the project’s most significant contribution, ensuring that developers not only receive immediate performance gains but also develop the long-term skills necessary to write efficient SQL and bridge the gap between database theory and daily industry practice.

Chapter 7

Future Scope

While the current implementation of Optimix provides a robust framework for SQL query optimization, several directions offer substantial potential for extending the system's technical capabilities and operational reach. The most immediate priority for future development is the expansion of multi-database engine support beyond the current MySQL 8+ focus. The system's layered architecture is specifically designed to separate database-specific metadata acquisition from its core optimization logic. By implementing new versions of the StatisticsCollector module, the system could integrate with PostgreSQL, Microsoft SQL Server, and SQLite. This would involve developing dialect-specific pattern variants to handle unique syntax and execution characteristics without requiring modifications to the fundamental cost-based or join-ordering algorithms.

Another significant area for growth is the integration of a machine learning layer to enhance selectivity estimation. Currently, the system utilizes fixed heuristics, such as 0.33 for range predicates and 0.10 for LIKE patterns, which may not always reflect skewed data distributions. A more advanced approach would involve a runtime cardinality feedback loop that compares estimated row counts against actual execution counts. This data could be used to train a lightweight, per-schema model capable of providing highly accurate, data-driven selectivity predictions, similar to the feedback mechanisms found in high-end commercial optimizers.

From an algorithmic perspective, the system could be extended to support bushy join trees. While the current implementation focuses on left-deep trees to maintain computational efficiency, support for bushy trees would improve join ordering for specific query structures, particularly in star schemas where a central fact table joins to multiple large dimension tables. This enhancement would allow the optimizer to fully leverage its $O(3^N)$ dynamic programming core to explore the absolute globally optimal join sequence for highly complex analytical workloads.

Finally, the system's operational utility could be expanded through DevOps integration and collaborative features. Developing a command-line API would allow Optimix to be integrated into CI/CD pipelines or Git pre-commit hooks, enabling organizations to automatically detect query performance regressions before they are merged into production codebases.

Appendix

Abstract Syntax Tree (AST): A hierarchical tree representation of the syntactic structure of a SQL statement produced by JSqlParser. Each node represents a syntactic construct such as a table, column, or expression, serving as the foundational data structure for the optimization pipeline.

Cost Model: A mathematical framework used to estimate query execution effort by modeling I/O and CPU components. It calculates the total cost based on the sum of sequential page access costs and CPU tuple evaluation costs.

Dynamic Programming (DP): An algorithmic technique for solving complex problems by decomposing them into overlapping subproblems and utilizing memoization to store intermediate results. This is the core logic used by the Join Optimizer to reduce complexity from $N!$ to $O(3^N)$.

Join Order: The specific sequence in which tables are joined during query execution. While different join orders produce identical final result sets, they can result in vastly different intermediate result sizes and total execution times.

SARGable: An acronym for Search ARGument able, referring to a predicate that can be satisfied using a direct index seek. Non-SARGable predicates, such as functions applied to indexed columns, typically force inefficient full table scans.

Selectivity (σ): The estimated fraction of rows in a table that satisfy a given predicate, expressed as a value between 0 and 1. This metric is a primary driver in the cost-based selection of index scans versus full scans.

Semi-Join: A specialized join operation that returns rows from the left table only if at least one matching row exists in the right table. It optimizes performance by avoiding the production of duplicate left-table rows.

TPC-H: The Transaction Processing Performance Council Benchmark H, an industry-standard analytical query workload consisting of 22 complex queries. It was used in this project to validate the semantic correctness of all 40 optimization patterns.

Bibliography

- [1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in Proc. ACM SIGMOD Int. Conf. on Management of Data, Boston, MA, USA, 1979, pp. 23–34.
- [2] W. Kim, “On optimizing an SQL-like nested query,” ACM Trans. Database Syst., vol. 7, no. 3, pp. 443–469, Sep. 1982.
- [3] J. D. Ullman, Principles of Database and Knowledge-Base Systems, vol. 1. Rockville, MD, USA: Computer Science Press, 1988.
- [4] The PostgreSQL Global Development Group, PostgreSQL 16 Documentation: Chapter 14 — Performance Tips. [Online]. Available: <https://www.postgresql.org/docs/16/performance-tips.html>. [Accessed: Apr. 2026].
- [5] Oracle Corporation, MySQL 8.0 Reference Manual: Chapter 8 — Optimization. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/optimization.html>. [Accessed: Apr. 2026].
- [6] Oracle Corporation, Oracle Database SQL Tuning Guide, 19c Release 1. Redwood Shores, CA, USA: Oracle Corporation, 2019.
- [7] B. Mozafari, C. Curino, A. Jindal, and S. Madden, “DBSeer: Resource and performance prediction for building a next generation database cloud,” in Proc. 6th Biennial Conf. on Innovative Data Systems Research (CIDR), Asilomar, CA, USA, 2013.
- [8] T. Nadižin, S. Nadižin, and R. J. Miller, “QueryViz: Helping users understand SQL queries and their results,” in Proc. 16th Int. Conf. on Extending Database Technology (EDBT), Genoa, Italy, 2013, pp. 558–563.
- [9] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdžic, “Robust query processing through progressive optimization,” in Proc. ACM SIGMOD Int. Conf. on Management of Data, Paris, France, 2004, pp. 659–670.
- [10] SolarWinds, Database Performance Analyzer Product Brief. Austin, TX, USA: SolarWinds Worldwide LLC, 2024.
- [11] EverSQL by Aiven, EverSQL SQL Query Optimizer and Index Advisor. [Online]. Available: <https://www.eversql.com>. [Accessed: Apr. 2026].